

Classificazione di immagini con modelli preaddestrati

L.Verdoliva, D.Cozzolino

In questa esercitazione useremo la libreria `timm` (PyTorch Image Models) che mette a disposizione diverse reti pre-addestrate per la classificazione di immagini. Prima di iniziare, resettiamo la console e importiamo le librerie che useremo:

```
%reset -f
import matplotlib.pyplot as plt
import skimage.io as io
import torch
import timm
import torch.optim as optim
import torch.utils.data as data
```

1 Applicazione di modelli preaddestrati

In `timm` sono disponibili molte reti già addestrate su ImageNet (<http://www.image-net.org/>), un ampio dataset composto da più di 14 milioni di immagini annotate manualmente in mille categorie. Utilizziamo una rete preaddestrata su ImageNet per classificare l'immagine `'test_dog.jpg'`:

```
img = io.imread('test_dog.jpg')
plt.figure()
plt.imshow(img)
plt.axis('off')
plt.show()
```

Creiamo il modello preaddestrato con architettura Xception usando la funzione `timm.create_model( )`:

```
device = "cuda:0" if torch.cuda.is_available() else "cpu" # usa la GPU se e' disponibile
model_name = 'xception41_tf_in1k'
model = timm.create_model(model_name, pretrained=True).to(device)
print(model)
```

Durante l'addestramento del modello le immagini vengono normalizzate e ridimensionate (pre-processing), quindi è importante conoscere le dimensioni consentite come input di un modello quando lo si vuole testare. Di seguito il codice per realizzare tali operazioni.

```
from timm.data import resolve_model_data_config
from timm.data.transforms_factory import create_transform
config = resolve_model_data_config(model)
transform = create_transform(**config)
print(transform)
```

Notate che per il modello preaddestrato Xception il preprocessing include un ridimensionamento a 331 pixel, un ritaglio centrale di 229×299 pixel, e una normalizzazione dei valori nell'intervallo $[-1, 1]$. Analizziamo il comportamento di questa rete su un'immagine di test:

```

from torchvision.transforms.functional import to_tensor
x = transform(to_tensor(img)) # applichiamo il pre-processing
x = x.unsqueeze(0).to(device)
model.eval() # disabilita comportamenti specifici del training
with torch.no_grad():
    logits = model(x)
    probs = torch.softmax(logits, dim=1)
top5_prob, top5_index = torch.topk(probs[0], 5)

```

In '**top5_prob**' abbiamo le 5 probabilità più alte restituite dalla rete e in '**top5_index**' abbiamo gli indici delle relative classi. Con il seguente codice visualizziamo le 5 probabilità con la descrizione testuale di ogni classe:

```

from timm.data import ImageNetInfo
classes = ImageNetInfo("imagenet-1k").label_descriptions() # descrizioni delle 1000 classi

for index in range(5):
    print(f"{top5_prob[index].item():.3f}", classes[top5_index[index].item()])

```

Provate a eseguire lo stesso codice sulle immagini '**elefante.jpg**', '**computer.jpg**', '**baba.jpg**'. Provate anche altri modelli, ad esempio '**convnextv2_base.fcmae_ft_in1k**' e '**vit_base_patch32_clip_224.laion2b_ft_in1k**'. Per la lista completa dei modelli pre-addestrati disponibili eseguite `timm.list_models(pretrained=True)`, notate che solo i modelli che finiscono per '**in1k**' sono preaddestrati su ImageNet.

2 Fine-Tuning

Il *fine-tuning* è una tecnica che permette di classificare immagini anche quando il dataset disponibile non è sufficientemente grande. L'idea è addestrare una rete non partendo da pesi casuali, ma utilizzando i pesi già ottenuti da un pre-addestramento su un altro dataset, molto più grande di quello che si ha a disposizione. In particolare, si usano i pesi relativi ai primi strati, che estraggono feature comuni a molte immagini (feature di basso livello), e si riaddestrano gli strati successivi, che estraggono feature specifiche dell'applicazione in esame. È una forma molto semplice di transfer learning che risulta comunque molto efficace.

In questa esercitazione useremo il dataset **TrafficSigns**. L'obiettivo è identificare se nell'immagine è presente un segnale stradale di tipo: mandatory, prohibition o warning. Su Colab potete direttamente eseguire le seguenti istruzioni per scaricare e decomprimere il dataset:

```

!wget -q -c https://www.grip.unina.it/download/guide_TF/TrafficSigns.zip
!unzip -q -n TrafficSigns.zip

```

In questo modo avrete una cartella **TrafficSigns** con due sottocartelle, **train** e **test**, ognuna contenente le tre classi: mandatory, prohibition, e warning. Visualizziamo un'immagine per ogni tipo di segnale stradale:

```

img0 = io.imread('TrafficSigns/train/mandatory/img0065_00.png')
img1 = io.imread('TrafficSigns/train/prohibition/img2929_00.png')
img2 = io.imread('TrafficSigns/train/warning/img2470_00.png')

plt.figure(figsize=(15,5))
plt.subplot(1,3,1); plt.imshow(img0); plt.title('mandatory')
plt.subplot(1,3,2); plt.imshow(img1); plt.title('prohibition')
plt.subplot(1,3,3); plt.imshow(img2); plt.title('warning')
plt.show()

```

2.1 Definizione del modello

Usiamo la rete ConvNext V2 pre-addestrata su ImageNet:

```
num_classes = 3
model_name = 'convnextv2_base.fcmae_ft_in1k'
device = "cuda:0" if torch.cuda.is_available() else "cpu" # usa la GPU se e' disponibile
model = timm.create_model(model_name, pretrained=True, num_classes=num_classes).to(device)
```

Notate che abbiamo specificato il numero di classi nella funzione `create_model`. Questa sostituirà l'ultimo livello della rete in modo da fornire un numero di uscite pari al numero di classi. Per bloccare l'addestramento dei parametri, eccetto quelli dell'ultimo livello (classification head), modifichiamo il loro attributo booleano `requires_grad` nel seguente modo:

```
# Freeze everything
for p in model.parameters():
    p.requires_grad = False # disattiva calcolo del gradiente

# Unfreeze only classification head
for p in model.get_classifier().parameters():
    p.requires_grad = True # attiva calcolo del gradiente

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in model.parameters())
print(f'Parametri addestrabili: {trainable_params:,} / {total_params:,}')
```

Realizziamo anche il pre-processing relativo al modello pre-addestrato:

```
from timm.data import resolve_model_data_config
from timm.data.transforms_factory import create_transform
config = resolve_model_data_config(model)
transform = create_transform(**config) # preprocessing del modello
norm_mean = transform.transforms[-1].mean # mean della normalizzazione
norm_std = transform.transforms[-1].std # std della normalizzazione
```

2.2 Preparazione dei dati

Poiché le immagini sono già organizzate in cartelle diverse per ogni classe, per creare l'oggetto `Dataset` possiamo usare `torchvision.datasets.ImageFolder`:

```
from torchvision.datasets import ImageFolder
trainval_set = ImageFolder(root='TrafficSigns/train', transform=transform)
test_set = ImageFolder(root='TrafficSigns/test', transform=transform)

print(f'Whole Training set: {len(trainval_set)} immagini')
print(f'Test set: {len(test_set)} immagini')
```

Separiamo le 3840 immagini di training in un validation set (20%) e un training set (80%) usando la funzione `random_split`:

```
train_set, val_set = data.random_split(trainval_set, [0.8, 0.2])
print(f'Training set: {len(train_set)} immagini')
print(f'Validation set: {len(val_set)} immagini')
print(f'Test set: {len(test_set)} immagini')
```

Creiamo i `DataLoader` per ciascun set specificando il `batch_size`:

```
batch_size = 8
train_loader = data.DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader = data.DataLoader(val_set, batch_size=batch_size, shuffle=False)
test_loader = data.DataLoader(test_set, batch_size=batch_size, shuffle=False)
```

2.3 Addestramento

Definiamo la loss function, l'ottimizzatore e il numero di epoche:

```
loss_function = torch.nn.CrossEntropyLoss()
optimizer = optim.SGD(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4,
    momentum=0.9,
)
num_epochs = 5
```

Possiamo utilizzare un learning rate anche molto basso, poiché non partiamo da pesi random. Per comodità, definiamo una funzione per l'addestramento di un'epoca:

```
def train_one_epoch(model, loader, loss_function, optimizer, device):
    model.train() # abilita comportamenti specifici del training (es. dropout)
    cum_loss = 0.0
    num_images = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad() # (1) azzera i gradienti
        logits = model(images) # (2) applica la rete
        loss = loss_function(logits, labels) # (3) calcolo del valore di loss
        loss.backward() # (4) calcolo gradienti
        optimizer.step() # (5) aggiornamento dei pesi

        # Aggiornamento delle statistiche
        cum_loss += loss.item() * images.shape[0]
        num_images += images.shape[0]

    train_loss = cum_loss / num_images

    return train_loss
```

Definiamo poi un'altra funzione per valutare le prestazioni:

```
def evaluate(model, loader, loss_function, device):

    model.eval() # disabilita comportamenti specifici del training
    epoch_cum_loss = 0.0
    epoch_cum_correct = 0
    epoch_num_images = 0

    with torch.no_grad(): # disabilita il calcolo dei gradienti
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)
```

```

        logits = model(images)
        loss = loss_function(logits, labels)
        predicted = torch.argmax(logits, -1)
        epoch_cum_loss += loss.item() * images.shape[0]
        epoch_cum_correct += torch.sum(predicted==labels).item()
        epoch_num_images += images.shape[0]

    epoch_loss = epoch_cum_loss / epoch_num_images
    epoch_acc = epoch_cum_correct / epoch_num_images

    return epoch_loss, epoch_acc

```

Eseguiamo il training per le diverse epoche:

```

for epoch in range(num_epochs):
    # ---- Fase di TRAINING ----
    train_loss = train_one_epoch(model, train_loader, loss_function, optimizer, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss train: {train_loss:.4f}')

    # ---- Fase di VALIDAZIONE ----
    val_loss, val_acc = evaluate(model, val_loader, loss_function, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss val: {val_loss:.4f} Acc val: {val_acc:.4f}')

```

Notate che con 5 epoche riusciamo già ad ottenere un'accuratezza sui dati di validazione superiore all'85%. Dopo l'addestramento potete salvare i pesi della rete con le seguenti istruzioni:

```

model_path = 'best_resnet50_traffic_signs.pt'
torch.save(model.state_dict(), model_path)

```

Successivamente, potete caricare i pesi nel seguente modo:

```

model_path = 'best_resnet50_traffic_signs.pt'
model.load_state_dict(torch.load(model_path, map_location=device))

```

2.4 Analisi delle prestazioni

Valutiamo le prestazioni del modello sul test set:

```

test_loss, test_accuracy = evaluate(model, test_loader, loss_function, device)
print('Test loss:', test_loss)
print('Test accuracy:', test_accuracy)

```

Testiamo la rete anche su una singola immagine:

```

# Test su una singola immagine
ID = 3 # indice dell'immagine nel test set

img, label = test_set[ID]
x = img.unsqueeze(0) # aggiungiamo la dimensione batch

model.eval()
with torch.no_grad():
    logits = model(x.to(device))
    probs = torch.softmax(logits, 1) # probabilita' per ogni classe
    pred = torch.argmax(logits, 1).item() # classe predetta

probs = probs[0].cpu().numpy() # conversione da torch a numpy

```

```
img = norm_std * torch.permute(img, (1,2,0)) + norm_mean # normalizzazione inversa
img = img.cpu().numpy() # conversione da torch a numpy

plt.figure()
plt.subplot(1,2,1); plt.imshow(img)
plt.title(f'Etichetta vera: {label}')
plt.subplot(1,2,2);
plt.bar(test_set.classes, probs)
plt.title(f'Etichetta predetta: {pred}')
plt.grid() # visualizza la griglia
plt.show()
```

3 Data Augmentation

Possiamo applicare alle immagini di training varie operazioni random che permettono di aumentare l'insieme dei dati di training in modo artificiale (*data augmentation*). A tal fine, possiamo utilizzare le trasformazioni random definite nella libreria `torchvision.transforms` ed eventualmente combinarle tramite `torchvision.Compose`.

```
from torchvision import transforms
augmented_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.RandomAffine(
        degrees=10,
        translate=(0.05, 0.05),
        scale=(0.8, 1.2),
    ),
    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.05,
    ),
    transform, # preprocessing del modello
])
```

La sequenza di operazioni `augmented_transform` va passata alla funzione che crea il Dataset:

```
trainval_set = ImageFolder(root='TrafficSigns/train', transform=augmented_transform)
```

4 Esercizi proposti

1. *Architettura*. Provate il fine-tuning utilizzando altre reti pre-addestrate disponibili in `timm`, ad esempio `vit_small_plus_patch16_dinov3.lvd1689m`.
2. *Augmentation*. Provate ulteriori operazioni di augmentation. Trovate la lista di possibili operazioni al link <https://docs.pytorch.org/vision/0.8/transforms.html>
3. *Evitare l'over-fitting*. Addestrate per più epoche salvando il modello con la migliore accuratezza di validazione. Confrontate le prestazioni del modello finale con quelle del modello migliore salvato.